

# claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-ae86dfbb29be6287f.jsonl`  
- SHA-256: `ccae57482e0359276644a553546e5a0e4c96d9ed067f06f090564458030dce6d`  
- Source modified: `2026-05-31T04:18:54+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

## Transcript

### 1. user (2026-05-31T04:18:17.539Z)

## Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract\_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

**\*\*URL:\*\*** <https://huggingface.co/mistralai/Pixtral-12B-2409>

**\*\*Title:\*\*** mistralai/Pixtral-12B-2409 ? Hugging Face (Apache-2.0 weights)

**\*\*Found via:\*\*** Model-weights license minefield (commercial-safety, decisive) search

## Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
  - be a concrete, checkable statement (not vague generalities)
  - include a direct quote from the source as support
  - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

### 2. assistant (2026-05-31T04:18:20.036Z)

I'll fetch the Pixtral-12B-2409 page and extract key claims relevant to the research question.

### 3. assistant (2026-05-31T04:18:23.154Z)

I have the WebFetch tool loaded. Now let me fetch the Pixtral-12B-2409 page.

### 4. user (2026-05-31T04:18:34.634Z)

## Pixtral-12B-2409 Model Details

### (1) License

**\*\*Apache-2.0\*\*** - Explicitly stated: "License: apache-2.0"

### (2) Model Size / Parameter Count

- **\*\*12B parameter Multimodal Decoder + 400M parameter Vision Encoder\*\***
- Total: ~12.4B parameters

### (3) Supported Inference Frameworks

```
**Primary recommendation: vLLM (v0.6.2+)**
```python
from vllm import LLM
llm = LLM(model="mistralai/Pixtral-12B-2409", tokenizer_mode="mistral")
```
```

```
**Alternative: mistral-inference (v1.4.1+)**
```python
from mistral_inference.transformer import Transformer
from mistral_inference.generate import generate
model = Transformer.from_folder(mistral_models_path)
```
```

Also supports: vLLM server/client mode via OpenAI-compatible API

#### (4) Document Understanding & Structured Data Capabilities

The model card does **not** explicitly state OCR, table extraction, or structured data capabilities. However, benchmark performance suggests document understanding:

- **DocVQA (ANLS): 90.7** - document visual question answering
- **ChartQA (CoT): 81.8** - chart understanding

No explicit claims about multi-column tables or structured extraction are stated.

#### (5) Vision/Multimodal Architecture

- **Vision Encoder**: 400M parameters
- **Image Support**: "Supports variable image sizes"
- **Multiple Images**: Up to 5 images per message supported

```
```python
limit_mm_per_prompt={"image": max_img_per_msg} # max_img_per_msg = 5
```
```
- **Training**: "Natively multimodal, trained with interleaved image and text data"

#### (6) Release Date / Version

**Version date: 2409** (September 2024) - indicated in model name "Pixtral-12B-2409"

Blog post reference: <https://mistral.ai/news/pixtral-12b/>

#### (7) Benchmark Performance (Document/Chart/OCR Tasks)

| Benchmark                | Pixtral 12B | Notable Comparisons       |
|--------------------------|-------------|---------------------------|
| <b>DocVQA</b> (ANLS)     | <b>90.7</b> | vs Claude-3 Haiku: 74.6   |
| <b>ChartQA</b> (CoT)     | <b>81.8</b> | vs Gemini-1.5 Flash: 78.0 |
| <b>VQAv2</b> (VQA Match) | <b>78.6</b> | vs GPT-4o: 77.8           |

- Additional key benchmarks:**
- MMMU (CoT): 52.5
  - MathVista (CoT): 58.0
  - Sequence length: **128k tokens**

#### 5. assistant (2026-05-31T04:18:39.826Z)

I have the key details. The fetch was successful and the page is highly relevant. Let me assess and structure the findings.

This is a primary source ? the official Hugging Face model card published by mistralai (the model creator). The license claim (Apache-2.0) is decisive for the research question's commercial-safety constraint, and the document benchmark numbers (DocVQA, ChartQA) are directly relevant to bank-statement table parsing.

#### 6. user (2026-05-31T04:18:46.142Z)

Structured output provided successfully

## 7. assistant (2026-05-31T04:18:54.539Z)

Completed. The Pixtral-12B-2409 Hugging Face model card was fetched successfully and is a primary source (official model card from mistralai, the model creator).

Key findings relevant to the muneem research question:

- License (decisive): Weights are Apache-2.0 ? commercially safe for both code and weights, clearing the project's hard licensing constraint. This is notably cleaner than many competing VLMS that ship restrictive community/non-commercial weight licenses.
- VRAM tier: ~12.4B params (12B decoder + 400M vision encoder), so it fits the ~16-24 GB tier; quantized variants would be needed for the ~8-12 GB consumer tier (not addressed on this page).
- Python integration: vLLM (v0.6.2+) and mistral-inference (v1.4.1+), both local-friendly.
- Document capability: DocVQA 90.7 ANLS and ChartQA 81.8 (CoT) ? strong document/chart understanding, though the card makes no explicit claim about OCR, borderless tables, or structured transaction extraction. Those are inferred capabilities, not stated guarantees.
- Multimodal handling: variable/native image sizes, up to 5 images per message, 128k context ? workable for feeding multi-page statement images.

License uncertainty: none on the weights themselves (Apache-2.0 is explicit). One caveat worth flagging to the parent agent ? the model card does not state reliability on dense financial tables; DocVQA/ChartQA are proxies, not direct evidence for the transposed-digit/dropped-row hallucination risks the research question raises.